

DSA Interview Preparation Roadmap

A Topic-Wise, Difficulty-Ordered Problem List — Basic → Easy → Medium → Hard

How to Use This Guide

This roadmap covers 12 core DSA topics. Within each topic, problems are ordered Basic → Easy → Medium → Hard so you build the underlying pattern before tackling harder variations of it. The third column names the exact technique each problem is meant to drill — if you can solve a problem but can't articulate that technique, revisit it before moving on.

- Don't skip Basic tier even if it looks trivial — it's there to lock in the raw mechanics (pointer movement, recursion structure, traversal order) before any twist is added.
- For every problem, attempt it cold for 20–30 minutes before looking at a solution. Struggling productively is where the pattern actually sticks.
- After solving, write a one-line note: 'this is an X pattern because Y.' That sentence is what you'll recall in an interview.
- Re-solve Medium/Hard problems from a topic a week later without notes. If you can't, the pattern isn't internalized yet.
- Search the exact problem title on GFG's Practice portal (or LeetCode, where most of these also exist) — titles are kept generic here since exact GFG URLs/slugs change over time.

Suggested Study Order

If you're building DSA from the ground up, this sequence front-loads patterns that reappear later (e.g. two pointers shows up inside linked lists, partitioning shows up inside quickselect/heaps):

1. Arrays & Hashing → 2. Sliding Window & Two Pointers → 3. Binary Search → 4. Recursion & Backtracking → 5. Linked List → 6. Stack & Queue → 7. Trees (Binary Trees) → 8. Binary Search Tree (BST) → 9. Heap / Priority Queue → 10. Greedy → 11. Graphs → 12. Dynamic Programming

Contents

1. Arrays & Hashing
2. Sliding Window & Two Pointers
3. Binary Search
4. Recursion & Backtracking
5. Linked List
6. Stack & Queue
7. Trees (Binary Trees)
8. Binary Search Tree (BST)
9. Heap / Priority Queue
10. Greedy
11. Graphs
12. Dynamic Programming

1. Arrays & Hashing

The foundation of almost every interview round. Master prefix sums, frequency maps, and in-place array manipulation before moving on.

Tier	Problem	Pattern / What It Builds
Basic	Largest Element in an Array	Single-pass linear scan
Basic	Second Largest Element in an Array	Single-pass tracking two values
Basic	Reverse an Array	In-place two-pointer swap
Basic	Rotate Array by D Places	Reversal algorithm / cyclic rotation
Basic	Move All Zeroes to the End	In-place partitioning
Easy	Check if Pair with Given Sum Exists (Two Sum)	Hashing for O(n) lookup
Easy	Find All Duplicates in an Array	Hashmap / frequency counting
Easy	Union and Intersection of Two Arrays	Hashset operations
Easy	Missing Number in an Array	Sum / XOR trick
Easy	Majority Element ($> n/2$ times)	Moore's Voting Algorithm
Easy	Frequency of Array Elements	Hashmap counting
Medium	Kadane's Algorithm (Maximum Subarray Sum)	Running-sum greedy/DP
Medium	Sort an Array of 0s, 1s, 2s (Dutch National Flag)	Three-pointer partitioning
Medium	Rearrange Array in Alternating Positive/Negative	Two-pointer rearrangement
Medium	Next Permutation	Lexicographic array manipulation
Medium	Longest Consecutive Subsequence	Hashset + sequence building
Medium	Subarray with Given Sum (incl. Zero Sum Subarray)	Prefix sum + hashmap
Hard	Trapping Rain Water	Prefix/suffix max arrays
Hard	Maximum Product Subarray	Running max/min tracking
Hard	Count Inversions in an Array	Merge-sort based counting
Hard	4Sum	Sorting + two-pointer + hashing
Hard	Median of Two Sorted Arrays of Different Sizes	Merge / binary-search hybrid

2. Sliding Window & Two Pointers

Almost every 'subarray/substring with a condition' question reduces to a fixed or variable window, or two pointers moving from opposite ends.

Tier	Problem	Pattern / What It Builds
Basic	Maximum Sum Subarray of Size K	Fixed-size window
Basic	Pair with Given Sum in a Sorted Array	Two pointers from both ends
Basic	First Negative Number in Every Window of Size K	Fixed window + deque
Easy	Smallest Subarray with Sum Greater than X	Variable-size window
Easy	Longest Substring Without Repeating Characters	Variable window + hashset
Easy	Container With Most Water	Two pointers on width/height
Easy	Remove Duplicates from a Sorted Array (in-place)	Two-pointer compaction
Medium	Sliding Window Maximum	Monotonic deque
Medium	Longest Substring with K Distinct Characters	Variable window + frequency map
Medium	Minimum Window Substring	Variable window + character-count matching
Medium	3Sum (Triplet Sum)	Sorting + two pointers
Medium	Subarrays with Sum Exactly K	Prefix sum + hashmap
Hard	Minimum Window Containing All Characters of Another String	Advanced variable window
Hard	Sliding Window Median	Two heaps / balanced structure
Hard	Find All Anagrams in a String	Fixed window + frequency matching
Hard	Count Subarrays with Exactly K Distinct Elements	$\text{atMost}(k) - \text{atMost}(k-1)$ window trick

3. Binary Search

Goes far beyond 'find element in sorted array' — the real skill is binary searching on the answer space itself.

Tier	Problem	Pattern / What It Builds
Basic	Binary Search (Standard)	Classic implementation
Basic	Search Insert Position / Floor in a Sorted Array	Boundary search
Basic	First and Last Occurrence of an Element	Two binary searches
Easy	Square Root of a Number (Integer)	Binary search on answer space
Easy	Count Occurrences of an Element in a Sorted Array	First/last occurrence difference
Easy	Search in a Sorted Array of Unknown Size	Exponential + binary search
Medium	Search in a Rotated Sorted Array	Modified binary search
Medium	Find the Minimum Element in a Rotated Sorted Array	Pivot detection
Medium	Find Peak Element	Binary search on unsorted-but-structured array
Medium	Allocate Minimum Number of Pages	Binary search on the answer
Medium	Painter's Partition Problem	Binary search on the answer
Medium	Search an Element in a Sorted 2D Matrix	Binary search on a matrix
Hard	Median of Two Sorted Arrays	Binary search on partitions
Hard	Kth Element of Two Sorted Arrays	Binary search merge
Hard	Aggressive Cows Problem	Binary search on answer (maximize the minimum)
Hard	Matrix Median	Binary search with rank counting

4. Recursion & Backtracking

Build comfort with the recursion tree first (subsets, permutations), then layer on pruning to turn brute force into backtracking.

Tier	Problem	Pattern / What It Builds
Basic	Print All Subsets of a Set	Recursive inclusion/exclusion
Basic	Tower of Hanoi	Classic recursion
Basic	Generate All Binary Strings of Length N	Recursive choice tree
Easy	Permutations of a String/Array	Recursive swap/backtrack
Easy	Subsets II (With Duplicates)	Backtracking with duplicate skipping
Easy	Letter Case Permutation	Recursive branching
Medium	N-Queen Problem	Constraint backtracking
Medium	Rat in a Maze	Grid backtracking
Medium	Combination Sum	Backtracking with pruning
Medium	Word Break (Backtracking Version)	Recursion + memoization
Medium	Generate Parentheses	Backtracking with validity constraints
Medium	Sudoku Solver	Constraint-satisfaction backtracking
Hard	M-Coloring Problem (Graph Coloring)	Backtracking on graphs
Hard	Word Search II (Backtracking + Trie)	Grid search with trie pruning
Hard	Palindrome Partitioning	Backtracking with palindrome checks
Hard	Knight's Tour Problem	Hamiltonian-path backtracking

5. Linked List

Pointer manipulation muscle memory. Fast-slow pointers and dummy-node tricks solve most of this category.

Tier	Problem	Pattern / What It Builds
Basic	Insert/Delete a Node in a Linked List	Pointer manipulation basics
Basic	Reverse a Linked List	Iterative/recursive reversal
Basic	Find the Middle of a Linked List	Fast-slow pointers
Easy	Detect a Loop in a Linked List	Floyd's cycle detection
Easy	Find the Length of a Loop in a Linked List	Cycle measurement
Easy	Nth Node from the End of a Linked List	Fast-slow pointer offset
Easy	Remove Duplicates from a Sorted Linked List	Single-pass pointer cleanup
Medium	Reverse a Linked List in Groups of K	Segmented reversal
Medium	Detect and Remove a Loop in a Linked List	Floyd's + pointer repair
Medium	Add Two Numbers Represented by Linked Lists	Digit-wise traversal
Medium	Intersection Point of Two Linked Lists	Two-pointer alignment
Medium	Clone a Linked List with a Random Pointer	Hashmap-based cloning
Medium	Flatten a Multilevel Linked List	Recursive merge
Hard	Merge K Sorted Linked Lists	Divide & conquer / heap merge
Hard	LRU Cache Design	Doubly linked list + hashmap
Hard	Sort a Linked List Using Merge Sort	Recursive merge sort on a list
Hard	Reverse Alternate K Nodes in a Linked List	Segmented recursive reversal

6. Stack & Queue

Monotonic stacks unlock a huge family of 'next greater / span / histogram' questions once you recognize the pattern.

Tier	Problem	Pattern / What It Builds
Basic	Implement a Stack Using an Array	Basic stack operations
Basic	Implement a Queue Using an Array	Basic queue operations
Basic	Implement Stack Using Two Queues / Queue Using Two Stacks	Structural conversion
Easy	Valid Parentheses (Balanced Brackets)	Stack-based matching
Easy	Next Greater Element	Monotonic stack
Easy	Implement a Min Stack	Auxiliary-stack tracking
Easy	Evaluate a Postfix/Prefix Expression	Stack-based evaluation
Medium	Largest Rectangle in a Histogram	Monotonic stack
Medium	Stock Span Problem	Monotonic stack
Medium	The Celebrity Problem	Stack / two-pointer elimination
Medium	Sliding Window Maximum (Deque-Based)	Monotonic deque
Medium	Design a Stack That Supports getMin() in O(1)	Auxiliary tracking
Hard	Trapping Rain Water Using a Stack	Monotonic stack
Hard	Asteroid Collision	Stack simulation
Hard	Maximum of All Subarrays of Size K	Deque optimization
Hard	Remove K Digits to Form the Smallest Number	Monotonic stack greedy
Hard	Basic Calculator (Expression Evaluation with Parentheses)	Stack-based parsing

7. Trees (Binary Trees)

Recursive thinking is everything here — most tree problems are 'do something at this node, combine results from children.'

Tier	Problem	Pattern / What It Builds
Basic	Inorder / Preorder / Postorder Traversal	Recursive traversal basics
Basic	Level Order Traversal	BFS on trees
Basic	Height of a Binary Tree	Recursive depth calculation
Easy	Diameter of a Binary Tree	Recursive height + tracking
Easy	Check if a Binary Tree Is Balanced	Height-based recursion
Easy	Mirror / Invert a Binary Tree	Recursive swap
Easy	Check if Two Trees Are Identical	Recursive comparison
Medium	Lowest Common Ancestor in a Binary Tree	Recursive path finding
Medium	Top View / Bottom View of a Binary Tree	BFS with horizontal distance
Medium	Vertical Order Traversal	BFS/DFS with coordinate mapping
Medium	Zig-Zag (Spiral) Level Order Traversal	BFS with direction toggling
Medium	Construct a Binary Tree from Inorder and Preorder	Recursive construction
Medium	Maximum Path Sum in a Binary Tree	Recursive DP on trees
Hard	Serialize and Deserialize a Binary Tree	Encoding/decoding traversal
Hard	Burning Tree Problem	BFS from a target node
Hard	Boundary Traversal of a Binary Tree	Combined traversal logic
Hard	Largest BST Subtree in a Binary Tree	Recursive validation + size tracking

8. Binary Search Tree (BST)

Use the BST ordering property to your advantage — *inorder traversal gives you a sorted sequence for free.*

Tier	Problem	Pattern / What It Builds
Basic	Insert a Node in a BST	Recursive/iterative insertion
Basic	Search in a BST	Recursive search using ordering
Basic	Delete a Node in a BST	Recursive deletion with successor handling
Easy	Check if a Tree Is a Valid BST	Recursive range validation
Easy	Find the Kth Smallest/Largest Element in a BST	Inorder traversal
Easy	Floor and Ceil in a BST	Recursive boundary search
Medium	Convert a Sorted Array to a Balanced BST	Recursive midpoint construction
Medium	Lowest Common Ancestor in a BST	Property-based traversal
Medium	Pair with Given Sum in a BST	Inorder + two pointers / hashing
Medium	BST Iterator (Inorder Successor on the Fly)	Controlled stack-based traversal
Medium	Convert a BST to a Sorted Doubly Linked List	Inorder + pointer rewiring
Hard	Merge Two BSTs	Inorder merge + reconstruction
Hard	Fix Two Swapped Nodes of a BST	Inorder violation detection
Hard	Largest BST in a Binary Tree (Mixed Tree)	Recursive validity + size
Hard	Inorder Successor and Predecessor in a BST	Boundary-tracking recursion

9. Heap / Priority Queue

Reach for a heap whenever you need the running min/max/Kth element without fully sorting the data.

Tier	Problem	Pattern / What It Builds
Basic	Build a Max/Min Heap from an Array	Heapify basics
Basic	Heap Sort	In-place heap-based sorting
Basic	Kth Largest Element in an Array	Heap-based selection
Easy	Check if an Array Represents a Min Heap	Structural validation
Easy	Kth Largest Element in a Stream	Running heap maintenance
Easy	Convert a Min Heap to a Max Heap	Heapify restructuring
Medium	Merge K Sorted Arrays	Min-heap merging
Medium	Top K Frequent Elements	Frequency map + heap
Medium	Sort a Nearly Sorted (K-Sorted) Array	Bounded heap of size K
Medium	Find the Median in a Running Stream	Two-heap technique
Medium	Rearrange Characters in a String (No Two Adjacent Same)	Max-heap greedy
Hard	Minimum Cost to Connect All Ropes	Greedy with min-heap
Hard	K Closest Points to the Origin	Max-heap of size K
Hard	Smallest Range Covering Elements from K Lists	Min-heap with pointers
Hard	Maximum Sum Combination from Two Arrays	Heap-based pair generation

10. Greedy

The hard part of greedy is proving the local choice is globally optimal — practice spotting 'sort + pick' structures.

Tier	Problem	Pattern / What It Builds
Basic	Activity Selection Problem	Sort by finish time + greedy pick
Basic	Minimum Number of Coins for a Value	Greedy coin selection
Basic	Maximum Meetings in One Room	Sort + greedy
Easy	Job Sequencing Problem	Sort by deadline/profit + greedy
Easy	Fractional Knapsack	Value/weight ratio greedy
Easy	Indian Coin Denominations / Assign Cookies	Greedy matching
Medium	N Meetings in One Room	Interval scheduling greedy
Medium	Minimum Number of Platforms Needed	Sort + two-pointer greedy
Medium	Huffman Coding	Greedy + min-heap
Medium	Minimum Swaps for Bracket Balancing	Greedy counting
Medium	Shortest Job First (Minimum Time to Complete Jobs)	Sort + greedy scheduling
Hard	Job Scheduling with Deadlines and Profits (Optimal)	Greedy + disjoint set / heap
Hard	Minimize the Maximum Difference Between Heights	Sort + greedy adjustment
Hard	Candy / Gas Station / Petrol Pump Problem	Circular greedy
Hard	Minimum Cost to Cut a Board into Squares	Greedy with sorted cuts

11. Graphs

Almost every graph problem is BFS, DFS, Union-Find, or a shortest-path algorithm wearing a different costume.

Tier	Problem	Pattern / What It Builds
Basic	BFS Traversal of a Graph	Queue-based traversal
Basic	DFS Traversal of a Graph	Recursive/stack-based traversal
Basic	Number of Islands (Connected Components in a Grid)	BFS/DFS on a grid
Easy	Detect a Cycle in an Undirected Graph	DFS / Union-Find
Easy	Detect a Cycle in a Directed Graph	DFS with recursion stack
Easy	Flood Fill Algorithm	DFS/BFS region filling
Easy	Topological Sort (Kahn's / DFS-Based)	Dependency ordering
Medium	Dijkstra's Shortest Path Algorithm	Greedy + min-heap
Medium	Bellman-Ford Algorithm	DP-based edge relaxation
Medium	Minimum Spanning Tree (Prim's & Kruskal's)	Greedy + Union-Find
Medium	Strongly Connected Components (Kosaraju's Algorithm)	DFS + reverse graph
Medium	Bipartite Graph Check	BFS/DFS two-coloring
Medium	Number of Provinces (Union-Find Application)	Disjoint Set Union
Hard	Floyd-Warshall Algorithm (All-Pairs Shortest Path)	DP on graphs
Hard	Articulation Points and Bridges in a Graph	DFS with discovery/low time
Hard	Word Ladder Problem	BFS on an implicit graph
Hard	Alien Dictionary (Topological Sort Application)	Graph construction + topo sort
Hard	Travelling Salesman Problem	DP with bitmasking

12. Dynamic Programming

The hardest topic for most people. Always identify the state first, then the recurrence — start 1D before attempting 2D/grid DP.

Tier	Problem	Pattern / What It Builds
Basic	Fibonacci Number Using DP	Memoization/tabulation basics
Basic	Climbing Stairs	1D DP
Basic	Coin Change (Minimum Coins)	1D DP with unbounded choices
Easy	0/1 Knapsack Problem	2D DP with a weight constraint
Easy	Subset Sum Problem	DP on boolean reachability
Easy	Longest Common Subsequence (LCS)	2D DP on strings
Easy	Longest Increasing Subsequence (LIS)	1D DP / $O(n \log n)$ variant
Medium	Edit Distance	2D DP on string transformations
Medium	Matrix Chain Multiplication	Interval DP
Medium	Longest Palindromic Subsequence	2D DP on strings
Medium	Partition Equal Subset Sum	Subset-sum DP variant
Medium	Unique Paths in a Grid (with Obstacles)	2D grid DP
Medium	House Robber Problem	1D DP with a skip constraint
Hard	Egg Dropping Puzzle	DP on attempts and floors
Hard	Word Break Problem (DP Version)	1D DP with dictionary lookups
Hard	Maximum Profit in Job Scheduling (Weighted Job Scheduling)	DP + binary search
Hard	Minimum Cuts for Palindrome Partitioning	Interval DP
Hard	Boolean Parenthesization	Interval DP with counting
Hard	Wildcard Pattern Matching	2D DP on string matching

Final Notes Before Interviews

- Aim for breadth before depth on the first pass: clear Basic + Easy across all 12 topics before grinding Hard problems in just one or two.
- Time-box yourself like a real interview on Medium/Hard problems (35–45 minutes) once you're past the first pass.
- Keep a running 'patterns I keep forgetting' list — review it the night before any interview instead of re-reading old solutions.
- Dynamic Programming and Graphs are typically the highest-leverage topics for product-based company interviews — weight your last two weeks toward them if time is limited.
- Mock interviews (explaining your approach out loud before coding) matter as much as problem count. Aim for at least a few timed, verbalized mocks before the real thing.